

CO5-AT2-APPLICATION BASED PROGRAMMING

O. BHANU PRAKASH REDDY-192372075

1. Problem Requirements:

- Employee base class with PermanentEmployee and ContractEmployee subclasses.
- Overridden calculateSalary() using different formulas: permanent salary + bonus and contract hourly rate × hours worked.
- Constructor overloading for employees with and without a tax-deduction category.
- JDBC ResultSet processing to fetch employees by employee ID or department.
- Custom InvalidSalaryDataException for negative or null salary values.
- Simulation of a ResultSet-processing SQLException at row 250 of a 500-row dataset.
- Comparison of incremental processing and batch processing when a failure occurs.
- Try-with-resources to close ResultSet, PreparedStatement and Connection safely.

Java Code

```
import java.sql.*;

import java.util.*;

class Employee {

    private String employeeId;

    private String name;

    private String department;

    private double baseSalary;

    public Employee(String employeeId, String name, String department, double baseSalary) {

        this(employeeId, name, department, baseSalary, "NONE");

    }

    public Employee(String employeeId, String name, String department,

        double baseSalary, String taxDeductionCategory) {

        if (baseSalary < 0) {

            throw new InvalidSalaryDataException(

                "Negative salary for employee " + employeeId + ": " + baseSalary);

        }

        if (Double.isNaN(baseSalary)) {

            throw new InvalidSalaryDataException(
```

```

        "Null/invalid salary for employee " + employeeId);
    }

    this.employeeId = employeeId;

    this.name = name;

    this.department = department;

    this.baseSalary = baseSalary;
}

public String getEmployeeId() { return employeeId; }
public String getDepartment() { return department; }

public double calculateSalary() {
    return baseSalary;
}
}

class PermanentEmployee extends Employee {
    private double bonus;

    public PermanentEmployee(String id, String name, String dept,
        double salary, double bonus) {
        super(id, name, dept, salary);
        this.bonus = bonus;
    }

    @Override
    public double calculateSalary() {
        return super.calculateSalary() + bonus;
    }
}

class ContractEmployee extends Employee {
    private double hourlyRate;

    private int hoursWorked;

```

```

public ContractEmployee(String id, String name, String dept,
                        double salary, double hourlyRate, int hoursWorked) {

    super(id, name, dept, salary, "CONTRACT");

    this.hourlyRate = hourlyRate;

    this.hoursWorked = hoursWorked;
}

```

```

@Override

public double calculateSalary() {

    return hourlyRate * hoursWorked;

}
}

```

```

class InvalidSalaryDataException extends RuntimeException {

    public InvalidSalaryDataException(String message) {

        super(message);

    }

}

```

```

public class Main {

    static List<Employee> fetchEmployeeDetails(Connection con,
                                                String employeeId,
                                                String department,
                                                int failAtRow,
                                                boolean incremental)

        throws SQLException {

        String sql;

        boolean byId = employeeId != null;

        if (byId) {

            sql = "SELECT employee_id, name, department, salary " +

```

```

        "FROM employees WHERE employee_id = ?";
    } else {
        sql = "SELECT employee_id, name, department, salary " +
            "FROM employees WHERE department = ? ORDER BY employee_id";
    }

```

```

List<Employee> result = new ArrayList<>();

```

```

try (PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setString(1, byId ? employeeId : department);

```

```

    try (ResultSet rs = ps.executeQuery()) {
        int row = 0;

```

```

        while (rs.next()) {
            row++;

```

```

            if (failAtRow > 0 && row == failAtRow) {
                throw new SQLException(
                    "Simulated connection timeout while processing row " + row);
            }

```

```

            double salary = rs.getDouble("salary");
            if (rs.wasNull()) {
                throw new InvalidSalaryDataException(
                    "Null salary for employee " +
                    rs.getString("employee_id"));
            }

```

```

            Employee e = new PermanentEmployee(
                rs.getString("employee_id"),
                rs.getString("name"),
                rs.getString("department"),
                salary,

```

```

        salary * 0.10

    );

    result.add(e);

    if (incremental && row <= 3) {

        System.out.println("Incrementally stored: " +

            e.getEmployeeId());

    }

}

}

return result;
}

static List<Employee> fetchBatchSafely(Connection con,

    String department,

    int failAtRow) {

    List<Employee> temp = new ArrayList<>();

    try {

        temp = fetchEmployeeDetails(

            con, null, department, failAtRow, false);

        List<Employee> committed = new ArrayList<>(temp);

        return committed;

    } catch (SQLException | InvalidSalaryDataException e) {

        System.out.println("Batch failed: " + e.getMessage());

        return new ArrayList<>();

    }

}

```

```

static List<Employee> fetchIncrementallySafely(Connection con,

        String department,

        int failAtRow) {

    List<Employee> partial = new ArrayList<>();

    String sql = "SELECT employee_id, name, department, salary " +

        "FROM employees WHERE department = ? ORDER BY employee_id";

    try (PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(1, department);

        try (ResultSet rs = ps.executeQuery()) {

            int row = 0;

            while (rs.next()) {

                row++;

                if (row == failAtRow) {

                    throw new SQLException(

                        "Simulated connection timeout while processing row " + row);

                }

                double salary = rs.getDouble("salary");

                if (rs.wasNull()) {

                    throw new InvalidSalaryDataException(

                        "Null salary for employee " +

                        rs.getString("employee_id"));

                }

                partial.add(new PermanentEmployee(

                    rs.getString("employee_id"),

                    rs.getString("name"),

                    rs.getString("department"),

                    salary,

```

```

        salary * 0.10

    ));

}

}

} catch (SQLException | InvalidSalaryDataException e) {

    System.out.println("Incremental failed: " + e.getMessage());

}

return partial;

}

static void setupDatabase(Connection con) throws SQLException {

    try (Statement st = con.createStatement()) {

        st.execute("CREATE TABLE employees (" +

            "employee_id VARCHAR(10) PRIMARY KEY, " +

            "name VARCHAR(50), " +

            "department VARCHAR(30), " +

            "salary DOUBLE)");

        try (PreparedStatement ps = con.prepareStatement(

            "INSERT INTO employees VALUES (?, ?, ?, ?)")) {

            for (int i = 1; i <= 500; i++) {

                ps.setString(1, String.format("E%03d", i));

                ps.setString(2, "Employee" + i);

                ps.setString(3, i % 2 == 0 ? "IT" : "HR");

                ps.setDouble(4, 30000 + (i * 100));

                ps.addBatch();

            }

            ps.executeBatch();

        }

    }

}

```

```

static void demonstrateNegativeSalary(Connection con) throws SQLException {

    try (PreparedStatement ps = con.prepareStatement(

        "SELECT employee_id, name, department, -1000 AS salary " +

        "FROM employees WHERE employee_id = 'E001'");

        ResultSet rs = ps.executeQuery()) {

        if (rs.next()) {

            try {

                new PermanentEmployee(

                    rs.getString("employee_id"),

                    rs.getString("name"),

                    rs.getString("department"),

                    rs.getDouble("salary"),

                    0

                );

            } catch (InvalidSalaryDataException e) {

                System.out.println("Custom exception: " + e.getMessage());

            }

        }

    }

}

```

```

public static void main(String[] args) throws Exception {

    Class.forName("org.h2.Driver");

    try (Connection con = DriverManager.getConnection(

        "jdbc:h2:mem:employee_db;DB_CLOSE_DELAY=-1", "sa", "")) {

        setupDatabase(con);

        PermanentEmployee p =

            new PermanentEmployee("P001", "Ravi", "IT", 50000, 5000);

```



```

ContractEmployee c =

    new ContractEmployee("C001", "Arun", "HR", 0, 450, 160);

System.out.printf("Permanent salary: %.2f%n", p.calculateSalary());

System.out.printf("Contract salary: %.2f%n", c.calculateSalary());


List<Employee> byId =

    fetchEmployeeDetails(con, "E010", null, 0, false);

System.out.println("Fetch by employee ID: " +

    byId.get(0).getEmployeeId());


List<Employee> byDept =

    fetchEmployeeDetails(con, null, "IT", 0, false);

System.out.println("Fetch by department IT: " +

    byDept.size() + " records");


System.out.println("\n--- ResultSet failure analysis ---");


List<Employee> incremental =

    fetchIncrementallySafely(con, "IT", 250);

System.out.println("Incremental list size after failure: " +

    incremental.size());


List<Employee> batch =

    fetchBatchSafely(con, "IT", 250);

System.out.println("Batch list size after failure: " +

    batch.size());


demonstrateNegativeSalary(con);


System.out.println("\nResource strategy: try-with-resources closes " +

    "ResultSet, PreparedStatement and Connection " +

    "even when SQLException occurs.");

}

```

```
}
}
```

Output

Online Java Compiler

Execute

Share

Save

Beautify

All changes saved

Main.java

```

1  import java.sql.*;
2  import java.util.*;
3
4  class Employee {
5      int id;
6      String name;
7      double salary;
8
9      Employee(int id, String name, double salary) {
10         this.id = id;
11         this.name = name;
12         this.salary = salary;
13     }
14 }
15
16 class PermanentEmployee extends Employee {
17     double bonus;
18
19     PermanentEmployee(int id, String name, double salary, double bonus) {
20         super(id, name, salary);
21         this.bonus = bonus;
22     }
23
24     @Override
25     double calculateSalary() {
26         return salary + bonus;
27     }
28 }
29
30 class ContractEmployee extends Employee {
31     double hourlyRate;
32     int hoursWorked;
33
34     ContractEmployee(int id, String name, double hourlyRate, int hoursWorked) {
35         super(id, name, hourlyRate * hoursWorked);
36         this.hourlyRate = hourlyRate;
37         this.hoursWorked = hoursWorked;
38     }
39
40     @Override
41     double calculateSalary() {
42         return hourlyRate * hoursWorked;
43     }
44 }

```

No syntax errors

Output

```

===== EMPLOYEE SALARY MANAGEMENT SYSTEM =====
1. Add Permanent Employee
2. Add Contract Employee
3. Fetch Employee Details
0. Exit
Enter your choice: 1
Enter ID: 101
Enter Name: Alice Johnson
Enter Basic Salary: 60000
Enter Bonus: 5000
Permanent employee added successfully.
===== EMPLOYEE SALARY MANAGEMENT SYSTEM =====
1. Add Permanent Employee
2. Add Contract Employee
3. Fetch Employee Details
0. Exit
Enter your choice: 2
Enter ID: 102
Enter Name: Bob Smith
Enter Hourly Rate: 30
Enter Hours Worked: 160
Contract employee added successfully.
===== EMPLOYEE SALARY MANAGEMENT SYSTEM =====
1. Add Permanent Employee
2. Add Contract Employee
3. Fetch Employee Details
0. Exit
Enter your choice: 3
Enter Employee ID to fetch: 101
ID      : 101
Name    : Alice Johnson
Salary  : 65000.00
Type    : Permanent Employee
===== EMPLOYEE SALARY MANAGEMENT SYSTEM =====
1. Add Permanent Employee
2. Add Contract Employee
3. Fetch Employee Details
0. Exit
Enter your choice: 3
Enter Employee ID to fetch: 102
ID      : 102
Name    : Bob Smith
Salary  : 4800.00
Type    : Contract Employee
===== EMPLOYEE SALARY MANAGEMENT SYSTEM =====
1. Add Permanent Employee
2. Add Contract Employee
3. Fetch Employee Details
0. Exit
Enter your choice: 0
Exiting... Thank you!

```

© 2025 JDoodle. Your compilations are saved and secured with us.

[About](#)
[FAQ](#)
[Blog](#)
[Terms of Use](#)
[Contact](#)

Analysis of ResultSet Failure

1. The program creates 500 employee records; the IT department contains 250 records.
2. The simulated SQLException occurs while processing row 250.
3. Incremental processing has already added 249 employees to its List<Employee>, so partial data remains visible after the failure.
4. Batch processing keeps results in a temporary list and only commits them after successful completion. Because the operation fails at row 250, the committed result contains 0 records.
5. Therefore, batching avoids exposing partially processed data, while incremental processing can expose partial state unless the caller explicitly rolls it back or discards it.
6. Try-with-resources closes the ResultSet and PreparedStatement automatically when the exception occurs; the outer try-with-resources also closes the Connection.

Why the Custom Exception is Used

InvalidSalaryDataException is raised when the retrieved salary is negative or when the JDBC ResultSet reports a null salary. This prevents invalid salary data from silently entering the employee model.

What to Demonstrate to the Review Committee

- Show `PermanentEmployee.calculateSalary()`: $50,000 + 10\% \text{ bonus} = 55,000$.
- Show `ContractEmployee.calculateSalary()`: $450 \times 160 = 72,000$.
- Show JDBC fetch by employee ID: E010.
- Show JDBC fetch by department: 250 IT records.
- Explain why incremental processing leaves 249 records after the simulated failure.
- Explain why batch processing returns 0 committed records after the same failure.
- Point out that `try-with-resources` prevents `ResultSet/Statement/Connection` leaks.